

DEPLOYMENT PORTFOLIO

TASK 4

Unit: SWE40006 - Software Deployment and Evolution

Submission date: 18 September 2026

Student name: Hong Hanh (Hayley) Nguyen

Student ID: 105005495

Declared Target Level: Task 4.4 High Distinction Level

TABLE OF CONTENTS

INTRODUCTION	3
TASK 4.1 - PASS LEVEL	4
Overview	4
Environment Setup	4
Docker Hub Account	5
Verification	5
TASK 4.2 - CREDIT LEVEL	7
Overview	7
Application Development	7
Containerisation	7
Local Deployment	8
Multi-Hosting	10
TASK 4.3 - DISTINCTION LEVEL	11
Overview	11
Application Development	11
Containerisation and Registry	12
Public Deployment	13
TASK 4.4 - HIGH DISTINCTION LEVEL	15
Overview	15
Application Development	15
Dockerfile	16
Building and Running with a Mounted Volume	17
Verifying the Container Lifecycle via Logs	18
TROUBLESHOOTING LOG	19
REPOSITORY	20

INTRODUCTION

This report documents the containerisation and deployment of applications using Docker. It covers all four target levels: installing and verifying the Docker environment using the official 'hello-world' image (Pass), containerising a Python web application and running it on a second Docker host through Docker Hub (Credit), developing a new web application with an optimised Dockerfile and deploying it to a public host over HTTP (Distinction), and containerising a non-web command-line tool using Docker best practices and a mounted host volume (High Distinction).

Each section explains the steps taken and the reasons behind the approach, as well as any issues encountered during the process. A consolidated troubleshooting log and links to the project repositories are also provided at the end of the report.

TASK 4.1 - PASS LEVEL

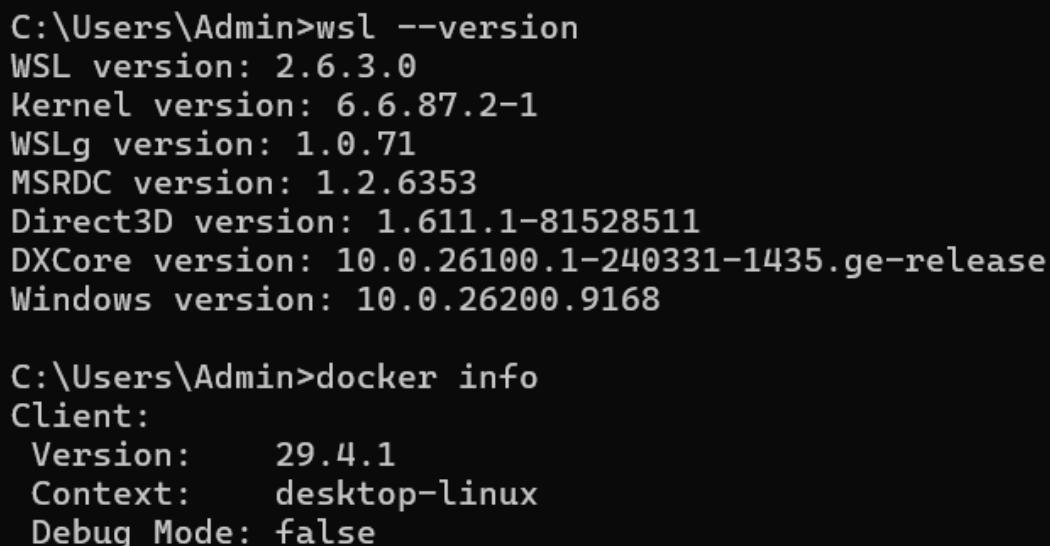
Overview

The Pass level involved setting up the Docker environment on the host machine and checking that the container runtime was working correctly. This included checking the WSL2 backend, confirming that Docker Desktop was installed and running, and making sure there was an active Docker Hub account. The official hello-world image was then pulled and run from Docker Hub to verify that Docker was working properly.

Environment Setup

Docker Desktop runs Linux containers on Windows using WSL2 (Windows Subsystem for Linux) as its backend. WSL2 and Docker Desktop had already been set up on the host machine during previous coursework, so they did not need to be installed again. Instead, the existing setup was checked to make sure it met the requirements. Details for the Docker environment used for this task:

- WSL version: 2.6.3.0 (kernel 6.6.87.2)
- Docker Engine: version 29.4.1 (client and server)
- Backend: WSL2 using the desktop-linux context
- Host operating system: Windows 11 (build 10.0.26200)

A terminal window with a black background and white text. The first command is 'C:\Users\Admin>wsl --version', which outputs: 'WSL version: 2.6.3.0', 'Kernel version: 6.6.87.2-1', 'WSLg version: 1.0.71', 'MSRDC version: 1.2.6353', 'Direct3D version: 1.611.1-81528511', 'DXCore version: 10.0.26100.1-240331-1435.ge-release', and 'Windows version: 10.0.26200.9168'. The second command is 'C:\Users\Admin>docker info', which outputs: 'Client:', 'Version: 29.4.1', 'Context: desktop-linux', and 'Debug Mode: false'.

```
C:\Users\Admin>wsl --version
WSL version: 2.6.3.0
Kernel version: 6.6.87.2-1
WSLg version: 1.0.71
MSRDC version: 1.2.6353
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.26200.9168

C:\Users\Admin>docker info
Client:
Version:      29.4.1
Context:      desktop-linux
Debug Mode:   false
```

Figure 4.1.1: Terminal output showing WSL and Docker version

Docker Hub Account

An active Docker Hub account (username: hehellyy) was used to sign in to Docker through Docker Desktop. This allows Docker to access images stored on Docker Hub and also allows images to be pushed to the registry.

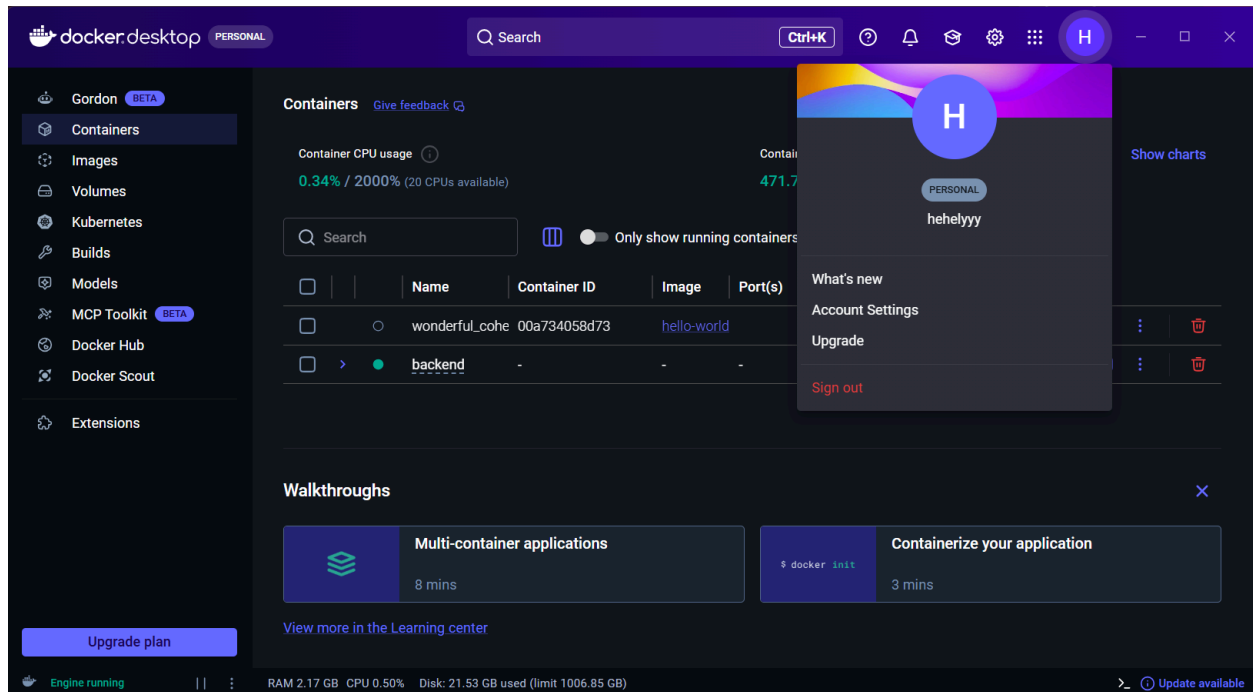


Figure 4.1.2: Docker Hub account signed in through Docker Desktop

Verification

The environment setup was confirmed to be running correctly, with both WSL2 and Docker Desktop operational as shown in Figure 4.1.1. Then, the official hello-world image was then run using “docker run hello-world”. Docker downloaded the image from Docker Hub, created and ran the container, and displayed the "Hello from Docker!" message. This confirmed that Docker was installed and able to successfully pull and run a container.

```
C:\Users\Admin>docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Figure 4.1.3: Docker run hello-world output showing the successful image pull

TASK 4.2 - CREDIT LEVEL

Overview

A simple Python web application was created using Flask, packaged into a custom Docker image, and run locally on port 8080. The image was then pushed to Docker Hub and run on a second Docker host, GitHub Codespaces, to confirm that it worked on another machine.

Application Development

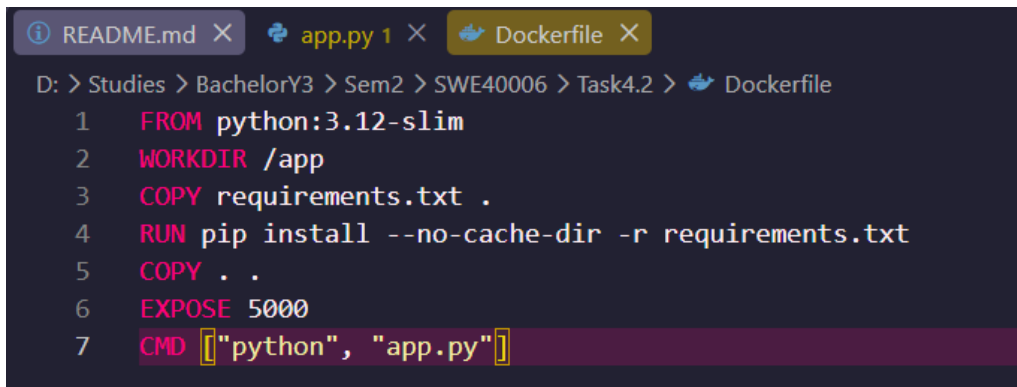
A basic web application was created using Flask, a lightweight Python web framework. The application contains a simple HTML page, with the sole purpose of confirming that it runs inside a Docker container. The project contained three files:

- app.py - the Flask application
- requirements.txt - the list of Python dependencies (Flask)
- Dockerfile - the instructions used to build the Docker image

The Flask application was configured to listen on 0.0.0.0 so that it could be accessed from outside the container. Using 127.0.0.1 would only allow connections from within the container.

Containerisation

The Dockerfile was created to package the application. Python 3.12 slim image was used as the base image. The application files were copied into the /app directory, Flask was installed from requirements.txt, and port 5000 was exposed for the web application. The CMD instruction starts the Flask application when the container runs.

A screenshot of a code editor window with three tabs: 'README.md', 'app.py 1', and 'Dockerfile'. The 'Dockerfile' tab is active, showing the following content:

```
D: > Studies > BachelorY3 > Sem2 > SWE40006 > Task4.2 > Dockerfile
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 EXPOSE 5000
7 CMD ["python", "app.py"]
```

Figure 4.2.1: Dockerfile for this application

Local Deployment

The Docker image was built from the project folder and named swe40006-task4. Then, the image was run with host port 8080 mapped to port 5000 inside the container. The application was accessed through <http://localhost:8080> and loaded successfully, confirming that the container was running correctly.

```
D:\Studies\BachelorY3\Sem2\SWE40006\Task4.2>docker build -t swe40006-task4 .
[+] Building 14.5s (11/11) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> => transferring dockerfile: 200B                               0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 2.9s
=> [auth] library/python:pull token for registry-1.docker.io       0.0s
=> [internal] load .dockerignore                                   0.1s
=> => transferring context: 2B                                     0.0s
=> [internal] load build context                                   0.1s
=> => transferring context: 525B                                   0.0s
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:78387bc3881b8 3.5s
=> => resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b8 0.1s
=> => sha256:1560cfed01dc4c72c07f789d860078d74466a1 12.12MB / 12.12MB 2.5s
=> => sha256:dbd0b7849e6c06b61e1fa6b7ffe053f246ce0075 4.27MB / 4.27MB 1.6s
=> => sha256:56235e4245636e401a28cf88944d543b29ee028ae3b6 249B / 249B 0.8s
=> => extracting sha256:dbd0b7849e6c06b61e1fa6b7ffe053f246ce007589062 0.4s
=> => extracting sha256:1560cfed01dc4c72c07f789d860078d74466a1f9a26b3 0.8s
=> => extracting sha256:56235e4245636e401a28cf88944d543b29ee028ae3b6c 0.0s
=> [2/5] WORKDIR /app                                              0.2s
=> [3/5] COPY requirements.txt .                                    0.1s
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt        5.4s
=> [5/5] COPY . .                                                  0.1s
=> exporting to image                                              1.9s
=> => exporting layers                                             1.2s
=> => exporting manifest sha256:43f3ea3bb05379f1ee78d0f9cda05f3009a65 0.0s
=> => exporting config sha256:2fa75c4e3e5d6e1e962f11ea22d6f84c3a49317 0.0s
=> => exporting attestation manifest sha256:0f8948670fb705e7c0b02e2ab 0.1s
=> => exporting manifest list sha256:3159ad9dc5041e6a2e5f4bdb2ed40943 0.0s
=> => naming to docker.io/library/swe40006-task4:latest           0.0s
=> => unpacking to docker.io/library/swe40006-task4:latest        0.5s

D:\Studies\BachelorY3\Sem2\SWE40006\Task4.2>docker run -d -p 8080:5000 swe40006-task4
3bf7de8dfelbd9f1fa0374e2ad777c146ec1f7e09591c881a43120fde805c809
```

Figure 4.2.2: docker build output showing the image being built

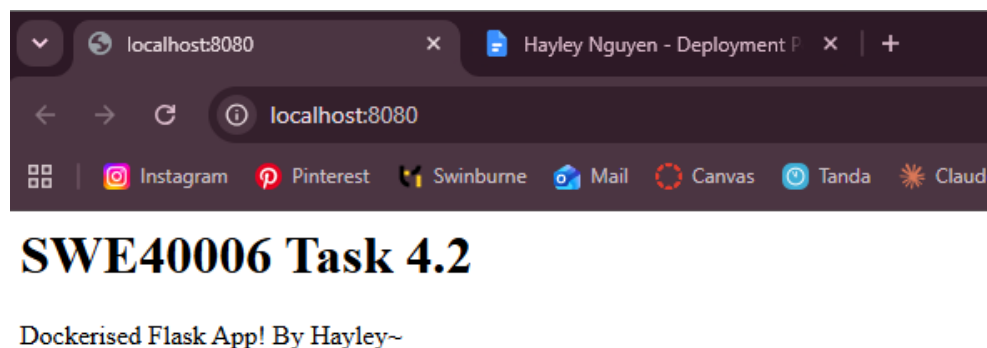


Figure 4.2.3: The containerised application running locally at localhost:8080

Docker was then authenticated using “docker login”. The local image was then tagged with the Docker Hub username and version: “docker tag swe40006-task4 hehellyyy/swe40006-task4:1.0”. The tagged image was then pushed to Docker Hub, and the image was successfully uploaded and appeared as a public repository on Docker Hub.

```
D:\Studies\BachelorY3\Sem2\SWE40006\Task4.2>docker push hehellyyy/swe40006-task4:1.0
The push refers to repository [docker.io/hehellyyy/swe40006-task4]
f57a69cda090: Pushed
9abb2a5bc68c: Pushed
11547dd03c24: Pushed
1560cfed01dc: Pushed
bfc8bc871da52: Pushed
6310eb16bf42: Pushed
dbd0b7849e6c: Pushed
56235e424563: Pushed
fbfc239ee9c7: Pushed
1.0: digest: sha256:3159ad9dc5041e6a2e5f4bdb2ed40943bbea8b80c74b80345f6a53ef3f99bc73 size: 856
```

Figure 4.2.4: Terminal output showing the image being uploaded to Docker Hub

The screenshot shows the Docker Hub interface for the repository **hehellyyy/swe40006-task4**. The page includes a header with the Docker Hub logo and a search bar. The repository details show it was last pushed about 1 hour ago, has a size of 48.8 MB, and 53 downloads. The 'Tags' section lists the tag **1.0** as an 'Image' type, pushed 'about 1 hour' ago. A sidebar on the right contains 'Docker commands' and a 'Public view' button. A 'buildcloud' advertisement is also visible on the right.

Repositories / swe40006-task4 / General

Using 0 of 1 private repositories.

hehellyyy/swe40006-task4

Last pushed about 1 hour ago • Repository size: 48.8 MB • ☆0 • ↓53

[Add a description](#) [Add a category](#)

Docker commands [Public view](#)

To push a new tag to this repository:

```
docker push hehellyyy/swe40006-task4:tagname
```

[General](#) [Tags](#) [Image Management](#) [Collaborators](#) [Webhooks](#) [Settings](#)

Tags

DOCKER SCOUT INACTIVE [Activate](#)

This repository contains 1 tag(s).

Tag	OS	Type	Pulled	Pushed
1.0		Image	less than 1 day	about 1 hour

[See all](#)

buildcloud

Build with **Docker Build Cloud**

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Figure 4.2.5: The image repository on Docker Hub

Multi-Hosting

To verify the image on a separate Docker host, GitHub Codespaces was used. The Codespace provided a separate Linux environment with Docker installed, without the local project files. The image was pulled from Docker Hub and run using: “docker run -d -p 8080:5000 hehellyy/swe40006-task4:1.0”. Docker automatically pulled the image from Docker Hub before starting the container. The application was then accessed through the Codespace's forwarded port URL ending in .app.github.dev and loaded successfully.

```
● @heheheyli →/workspaces/swe40006-task4 (main) $ docker images
docker run -d -p 8080:5000 --name task4-app hehellyy/swe40006-task4:1.0

IMAGE ID          DISK USAGE  CONTENT SIZE  EXTRA
Unable to find image 'hehellyy/swe40006-task4:1.0' locally
1.0: Pulling from hehellyy/swe40006-task4
1560cfed01dc: Pull complete
9abb2a5bc68c: Pull complete
11547dd03c24: Pull complete
bfc8c871da52: Pull complete
fbfc239ee9c7: Pull complete
56235e424563: Pull complete
6310eb16bf42: Pull complete
dbd0b7849e6c: Pull complete
f57a69cda090: Download complete
Digest: sha256:3159ad9dc5041e6a2e5f4bdb2ed40943bbea8b80c74b80345f6a53ef3f99bc73
Status: Downloaded newer image for hehellyy/swe40006-task4:1.0
98b8a7b56bb8558ed58ba55ba1e9d8468bc4ff79089609e22ea687d5e8b55b1d
```

Figure 4.2.6: The image pulled and running inside GitHub Codespaces

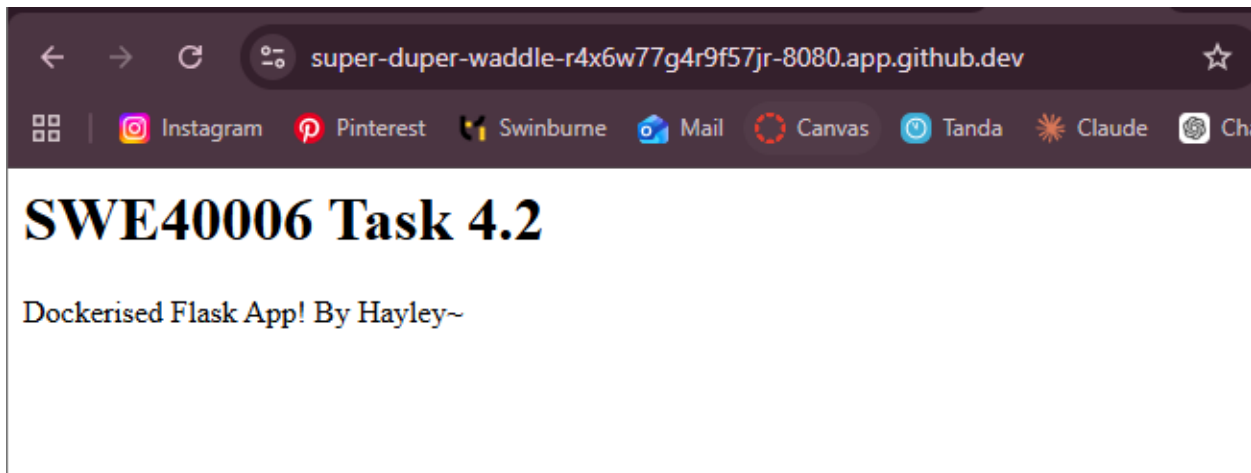


Figure 4.2.7: The containerised application running from Codespaces

TASK 4.3 - DISTINCTION LEVEL

Overview

A new web application was created using FastAPI for this task. An optimised Dockerfile was written and the application was deployed to a containerised environment where it is publicly accessible over an HTTP URL.

Application Development

A new web application was built using FastAPI, a modern Python web framework. FastAPI was chosen to make the application different from the Flask application used in the Credit level. The application is a simple "decision maker" tool where the user enters a dilemma and the application returns a randomised verdict. It used a JSON API endpoint and automatically generated interactive API documentation. The project contained three main files:

- main.py: the FastAPI application home page with API endpoint and logic
- requirements.txt: the dependencies (FastAPI and Uvicorn)
- Dockerfile: the build instructions

FastAPI does not include a web server for running the application by itself, so Uvicorn was included as the server. The application provides three points of access: a home page (/) with a form, a JSON endpoint (/api/decide) that returns the verdict as structured data, and an automatically generated documentation page (/docs).

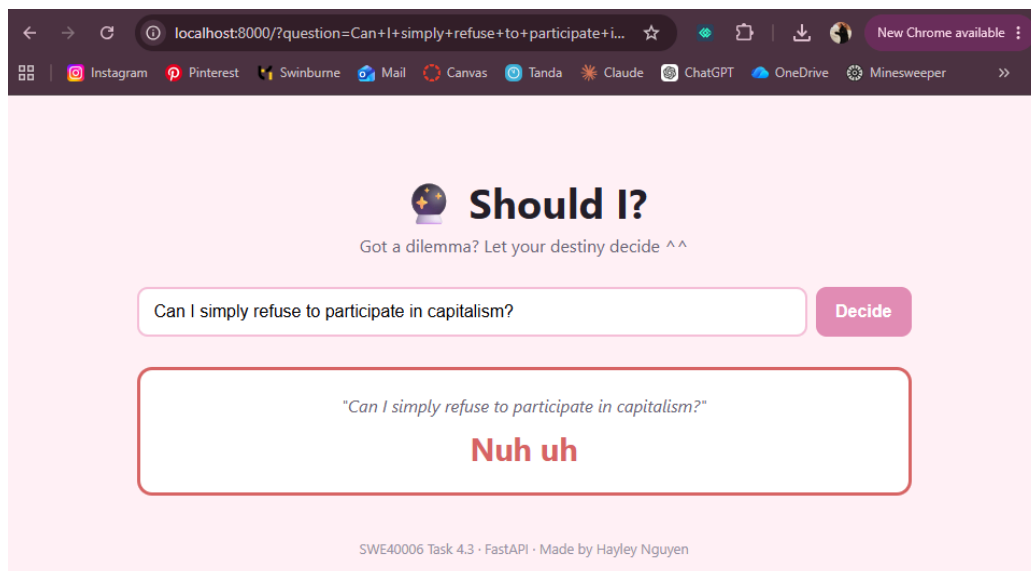
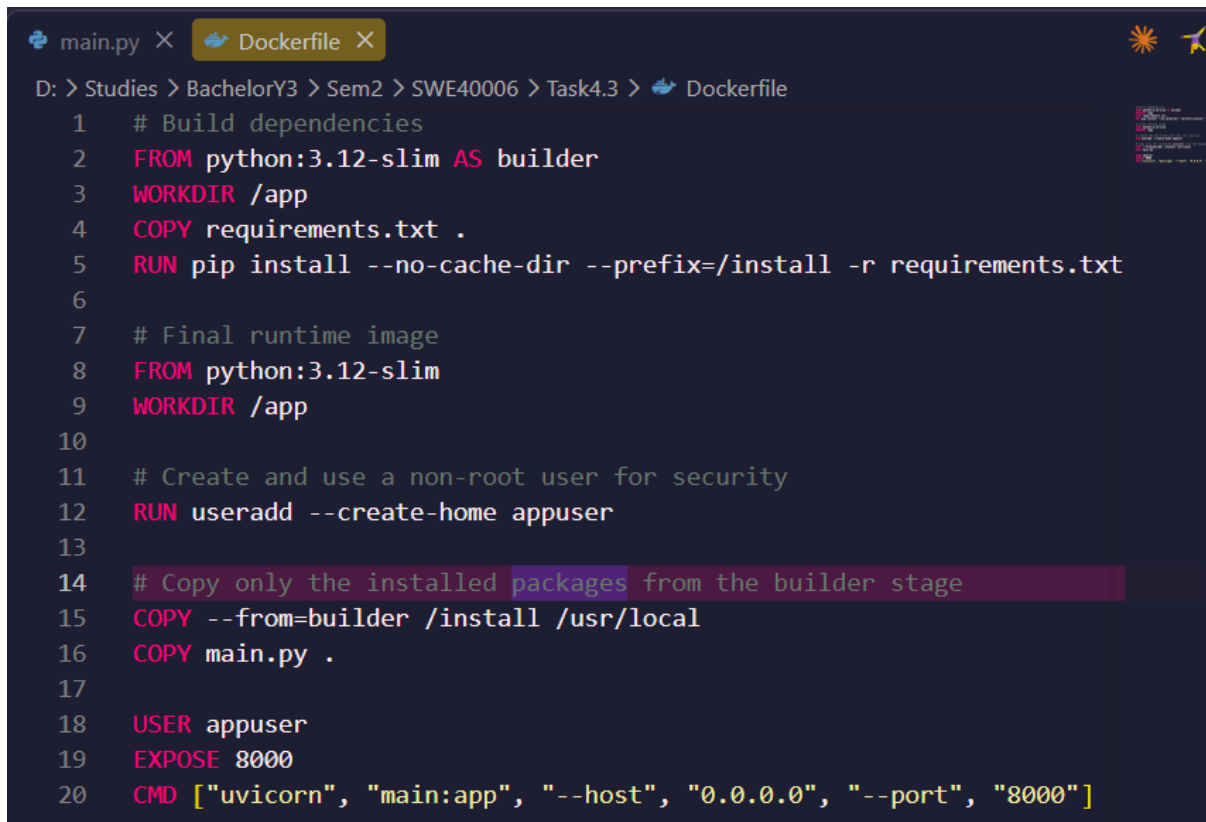


Figure 4.3.1: The FastAPI application running locally

Containerisation and Registry

For this task, a more optimised Dockerfile was written with several inclusions:

- Multi-stage build: The dependencies are installed in a separate builder stage, and only the installed packages are copied into the final image. The extra files from the installation process are left in the builder stage, which helps keep the final image smaller and cleaner.
- Non-root user: A separate account is created, and the application runs using this account instead of the root user. This improves security by limiting what the application can access if the container is compromised.
- Layer caching: requirements.txt is copied and installed before the application code. This means Docker can reuse the dependency layer when only the application code changes, instead of reinstalling the dependencies each time.
- --no-cache-dir: pip's download cache is not saved in the image, which helps reduce the final image size.



```
main.py × Dockerfile ×
D: > Studies > BachelorY3 > Sem2 > SWE40006 > Task4.3 > Dockerfile
1  # Build dependencies
2  FROM python:3.12-slim AS builder
3  WORKDIR /app
4  COPY requirements.txt .
5  RUN pip install --no-cache-dir --prefix=/install -r requirements.txt
6
7  # Final runtime image
8  FROM python:3.12-slim
9  WORKDIR /app
10
11 # Create and use a non-root user for security
12 RUN useradd --create-home appuser
13
14 # Copy only the installed packages from the builder stage
15 COPY --from=builder /install /usr/local
16 COPY main.py .
17
18 USER appuser
19 EXPOSE 8000
20 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Figure 4.3.2: The Dockerfile

The image was then built and tagged before being pushed to Docker Hub as a separate repository from the Credit application. This kept the two applications clearly separated.

```
D:\Studies\BachelorY3\Sem2\SWE40006\Task4.3>docker tag swe40006-task4-3 hehelyyy/task4.3-dilemma:1.0

D:\Studies\BachelorY3\Sem2\SWE40006\Task4.3>docker push hehelyyy/task4.3-dilemma:1.0
The push refers to repository [docker.io/hehelyyy/task4.3-dilemma]
75f01a037caf: Pushed
851fef6d6ad3: Pushed
11547dd03c24: Mounted from hehelyyy/swe40006-task4
1560cfed01dc: Mounted from hehelyyy/swe40006-task4
4a8ee414a5a2: Pushed
6310eb16bf42: Mounted from hehelyyy/swe40006-task4
56235e424563: Mounted from hehelyyy/swe40006-task4
dbd0b7849e6c: Mounted from hehelyyy/swe40006-task4
8e103bb5419d: Pushed
1.0: digest: sha256:0cfbff1ea4739d5b1c8db20de9279fba5f67968b68d2e3cd4ea78fcc9149fc94
size: 856
```

Figure 4.3.3: The image pushed to Docker Hub repository

Public Deployment

To make the application publicly accessible over HTTP, it was deployed to Render, a cloud platform that can run a container directly from a Docker Hub image. Render was chosen because it provides a free public HTTPS URL without requiring a payment method, and the Docker Hub image can be deployed with only minimal configuration.

A new Web Service was created on Render using the option to deploy an existing image from a registry. The service was connected to the hehelyyy/task4.3-dilemma:1.0 image and configured to use port 8000, which is the port the application listens on inside the container. After deployment, Render pulled the image, started the container, and provided a public URL. The application was then tested through the Render URL and loaded successfully in the browser. Users could submit a dilemma and receive a verdict as expected.

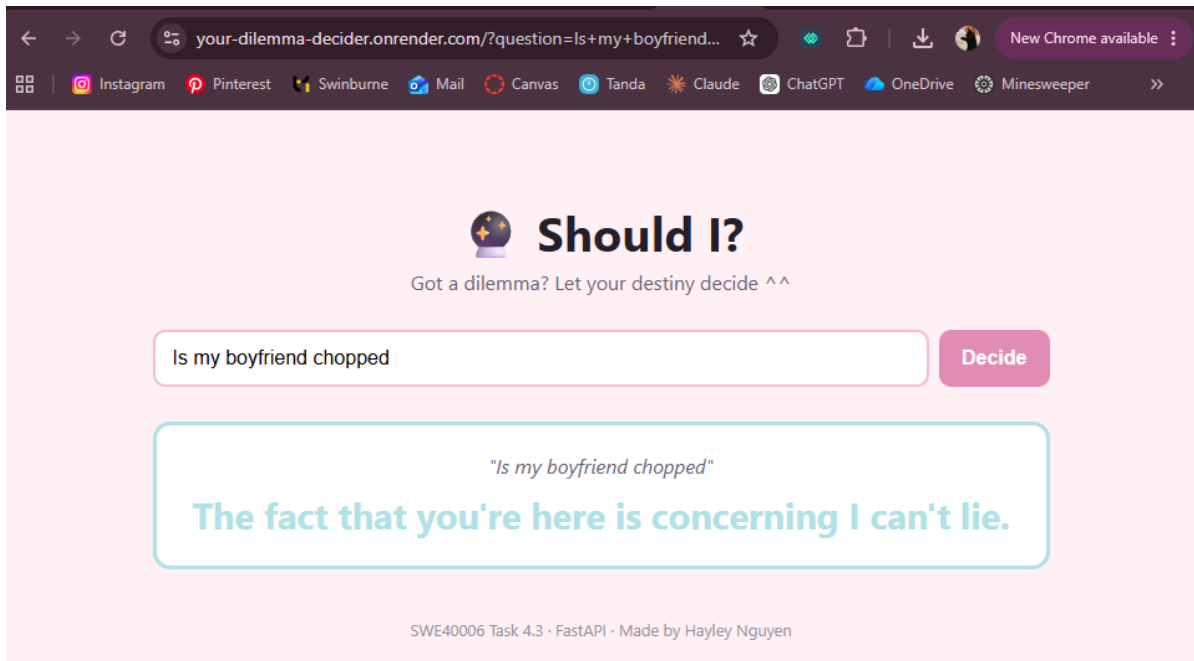


Figure 4.3.4: The application running at public Render URL

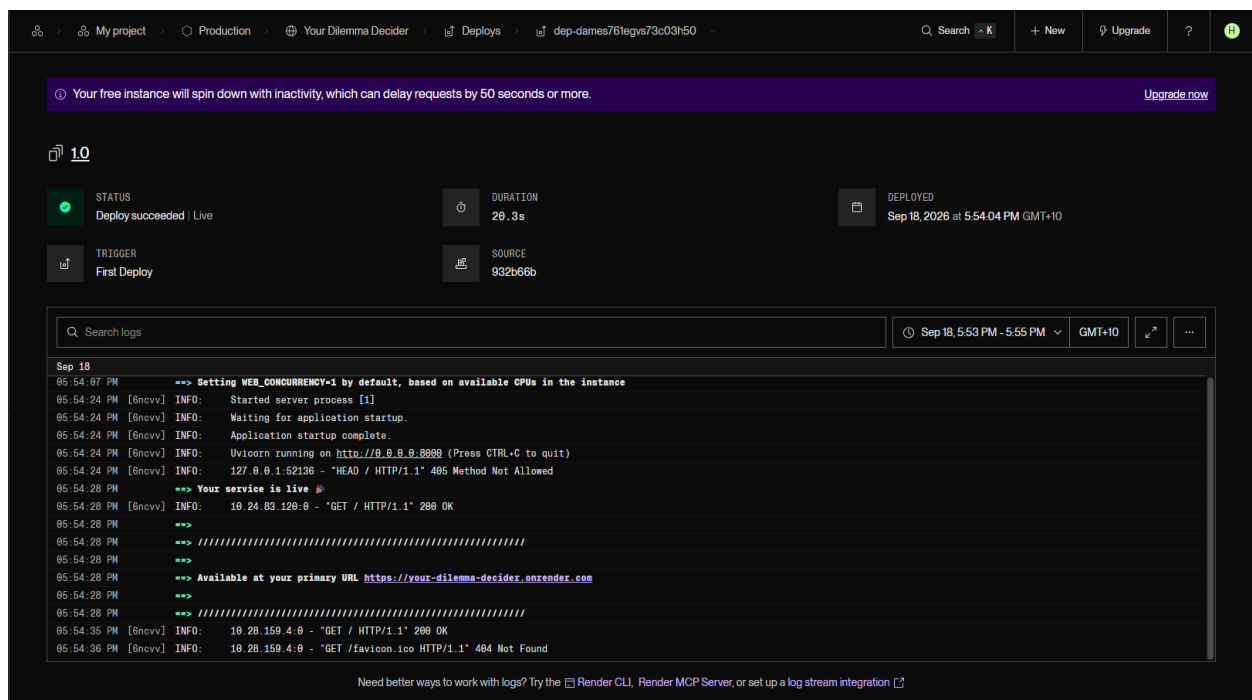


Figure 4.3.5: The live deployed service on Render dashboard

TASK 4.4 - HIGH DISTINCTION LEVEL

Overview

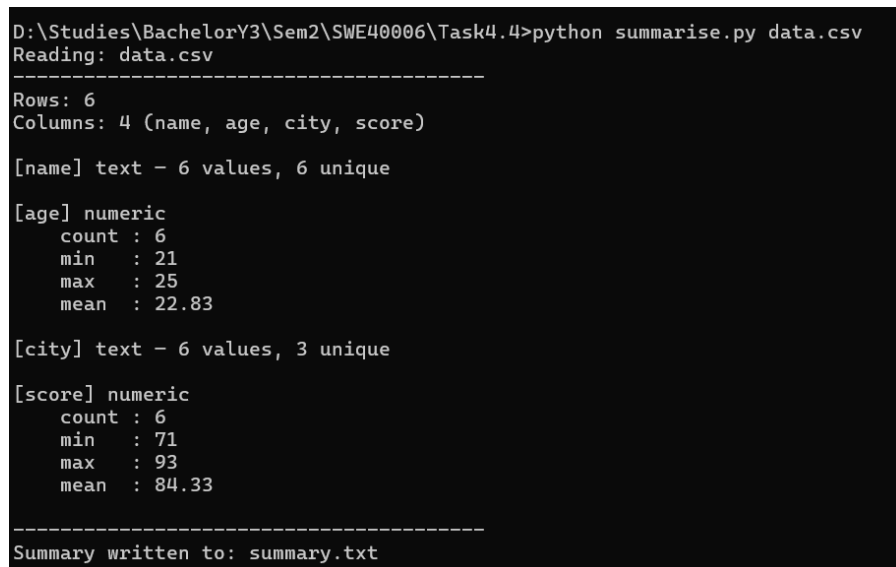
The application was a command-line data-processing tool written in Python. It was containerised using Docker and run against a real dataset provided at runtime through a mounted host volume, and the container logs were used to demonstrate its lifecycle. This shows that Docker is not limited to long-running web servers, but can also package short-lived tools that run a task and exit.

Application Development

A command-line tool was developed in Python to summarise data from a CSV file. It reads the input CSV file, prints a summary to the console, and saves the summary to a text file. For each column it reports the number of values and, where the column is numeric, its count, minimum, maximum, and mean; for text columns it reports the number of values and how many are unique. The tool uses only the Python standard library, so no external dependencies are required. It takes its input and output file paths as command-line arguments:

```
python summarise.py <input.csv> [output.txt]
```

This design works well for a Docker container because the tool performs one specific task, produces clear output, and then exits. This is different from a web application, which normally continues running and waits for requests.



```
D:\Studies\BachelorY3\Sem2\SWE40006\Task4.4>python summarise.py data.csv
Reading: data.csv
-----
Rows: 6
Columns: 4 (name, age, city, score)

[name] text - 6 values, 6 unique

[age] numeric
      count : 6
      min   : 21
      max   : 25
      mean  : 22.83

[city] text - 6 values, 3 unique

[score] numeric
      count : 6
      min   : 71
      max   : 93
      mean  : 84.33
-----
Summary written to: summary.txt
```

Figure 4.4.1: Application running, using a sample data file

Dockerfile

The Dockerfile written to package the tool applied best practices:

- Pinned slim base image: `python:3.12-slim` keeps the image small and pins a specific version so the build is consistent rather than changing when a newer version is released.
- Non-root user: the tool runs as `appuser` instead of `root`, following the principle of least privilege.
- Minimal image contents: only the Python script is copied in. Since the tool uses no external libraries, there is no dependency installation step, which keeps the image as small as possible.
- `ENTRYPOINT` instead of `CMD`: using `ENTRYPOINT` makes the container behave like the command itself, so any arguments passed to `docker run` are forwarded directly to the script.

No dataset was copied into the Docker image. This was intentional because the CSV file is provided at runtime through a mounted volume. This keeps the image generic and allows it to be reused with different CSV files.

```
main.py X summarise.py X Dockerfile X
D: > Studies > BachelorY3 > Sem2 > SWE40006 > Task4.4 > Dockerfile
1  FROM python:3.12-slim
2
3  # Run as a non-root user
4  RUN useradd --create-home appuser
5  USER appuser
6  WORKDIR /home/appuser
7
8  # Copy only the script, no dependencies needed
9  COPY summarise.py .
10
11 # Data is provided at runtime
12 ENTRYPOINT ["python", "summarise.py"]
13
```

Figure 4.4.2: Dockerfile for this task

Building and Running with a Mounted Volume

The image was built, and the container was then run with the host's current folder mounted into the container at /data. This was an important step because it demonstrates the use of host-mounted storage. The container itself does not contain any data but reads the input file from the mounted host folder and writes the output back to the same location. A real energy consumption dataset containing 35,040 rows was used as the input.

```
D:\Studies\BachelorY3\Sem2\SWE40006\Task4.4>docker build --no-cache -t swe40006-task4-4 .
[+] Building 2.2s (9/9) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 296B                             0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 0.9s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> CACHED [1/4] FROM docker.io/library/python:3.12-slim@sha256:78387bc 0.1s
=> => resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b82 0.1s
=> [internal] load build context                                0.1s
=> => transferring context: 2.38kB                               0.0s
=> [2/4] RUN useradd --create-home appuser                      0.4s
=> [3/4] WORKDIR /home/appuser                                  0.1s
=> [4/4] COPY summarise.py .                                    0.1s
=> exporting to image                                           0.4s
=> => exporting layers                                           0.2s
=> => exporting manifest sha256:c7cf98bd43ad763e1d837a1d4d10f5c8d6dab1 0.0s
=> => exporting config sha256:1cbe20390f2c2d09532a15ca79b413eacfdb0492 0.0s
=> => exporting attestation manifest sha256:4b5b85191c5dcf0f2ad084003a 0.0s
=> => exporting manifest list sha256:9bd7fd19372cd2d04728ae0ef2182f960 0.0s
=> => naming to docker.io/library/swe40006-task4-4:latest       0.0s
=> => unpacking to docker.io/library/swe40006-task4-4:latest    0.1s
```

Figure 4.4.3: The docker build output

```
D:\Studies\BachelorY3\Sem2\SWE40006\Task4.4>docker run --rm -v "%cd%/data" swe40006-task4-4 /data/data2.csv /data/summary.txt
Reading: /data/data2.csv
-----
Rows: 35040
Columns: 11 (date, Usage_kWh, Lagging_Current_Reactive_Power_kVarh, Leading_Current_Reactive_Power_kVarh, CO2(tCO2), Lagging_Current_Power_Factor, Leading_Current_Power_Factor, NSM, WeekStatus, Day_of_week, Load_Type)
[date] text - 35040 values, 35040 unique

[Usage_kWh] numeric
count : 35040
min   : 0
max   : 157.18
mean  : 27.39

[Lagging_Current_Reactive_Power_kVarh] numeric
count : 35040
min   : 0
max   : 96.91
mean  : 13.04

[NSM] numeric
count : 35040
min   : 0
max   : 85500
mean  : 42750.00

[WeekStatus] text - 35040 values, 2 unique
[Day_of_week] text - 35040 values, 7 unique
[Load_Type] text - 35040 values, 3 unique
-----
Summary written to: /data/summary.txt
```

Figure 4.4.4: The container reading the mounted dataset

The command is made up of the following parts:

- `-v "%cd%:/data"` - mounts the current host folder into the container as `/data` (the volume mount)
- `swe40006-task4-4` - specifies the Docker image being run.
- `/data/data2.csv /data/summary.txt` - the input and output paths passed to the tool, both inside the mounted folder
- `--rm` - automatically removes the container once it has finished, keeping the Docker environment clean

The container read the dataset, printed the full summary to the console, and reported that the summary had been written back out. All the columns were summarised correctly. Because the output file was written to the mounted `/data` path, it appeared in the host folder after the container exited, confirming that the volume mount worked in both directions.

Verifying the Container Lifecycle via Logs

To verify the container lifecycle, the tool was run again as a named container without the `--rm` flag, so that the container remained after exiting and its logs could be inspected.

```
D:\Studies\Bachelory3\Sem2\SWE40006\Task4.4>docker logs task44-run
Reading: /data/data2.csv
=====
Rows: 35040
Columns: 11 (date, Usage_kWh, Lagging_Current_Reactive_Power_kVarh, Leading_Current_Reactive_Power_kVarh, CO2(tCO2), Lagging_Current_Power_Factor, Leading_C
urrent_Power_Factor, NSM, WeekStatus, Day_of_week, Load_Type)
[date] text - 35040 values, 35040 unique

[Usage_kWh] numeric
count : 35040
min   : 0
max   : 157.18
mean  : 27.39

[Lagging_Current_Reactive_Power_kVarh] numeric
count : 35040
min   : 0
max   : 96.91
mean  : 13.04
```

Figure 4.4.5: The docker logs output

The docker logs command showed the full output the tool produced during its run, confirming that the container had started, executed the task, produced its output, and exited cleanly. The container was then removed.

TROUBLESHOOTING LOG

Task	Issue	Cause	Resolution
4.2	On Codespaces, docker login authenticated as an unexpected account (codespacesdev) rather than the intended Docker Hub account	GitHub Codespaces has its own built-in Docker credentials, which the client used by default	Ran docker logout followed by docker login -u hehelyyy to sign in with the correct Docker Hub account
	docker push failing with "push access denied"	Docker account not logged in after the account's password was changed	Ran docker logout followed by docker login -u hehelyyy to sign in
4.4	The container failed at runtime with python: can't open file '/home/appuser/summarise.py', and the build output showed no COPY step	The COPY summarise.py . line had been accidentally deleted from the Dockerfile, so the script was never added to the image	Restored the COPY summarise.py . line and rebuilt with docker build --no-cache

REPOSITORY

GitHub Repository: <https://github.com/heheheyli/swe40006-task4>

DockerHub Images:

- Task 4.2: <https://hub.docker.com/repository/docker/hehellyyy/swe40006-task4>
- Task 4.3: <https://hub.docker.com/repository/docker/hehellyyy/task4.3-dilemma>

Public URL:

- Task 4.3: <https://your-dilemma-decider.onrender.com/>